

Zenopsys Assignment: A Configurable Workflow Engine

Language / stack: Your choice. Any HTTP framework and any datastore (in-memory, SQLite, Postgres, ...) are acceptable. You can use any coding agent/tools to generate code

You own all the design decisions: the data model, the abstractions, the API shapes, the internal structure.

1. The problem

Build a service that lets a user **define** a workflow once and then **run** it many times, driving each run to completion through an API.

A workflow is a graph of tasks. A user authors a reusable **definition** (the template). They then **start** a run from that definition; the run executes task by task. Some tasks are completed by a human submitting data; some are resolved automatically by the engine based on data collected earlier. When a task finishes, the engine decides what runs next and advances the run, until the workflow reaches a terminal point and the run is complete.

Your job is to model this, expose it over an API, and make a run go all the way through — including a branch in the path that depends on data entered during the run.

2. Functional requirements

Authoring a workflow (the definition / template)

1. A user can create a workflow definition and fetch it back.
2. The system supports **three kinds of task**, and a definition may mix them:
 - **A data-entry task** completed by a human. It collects one or more inputs, and may be organized into **multiple ordered steps** that the human submits one at a time; the task is only finished once its final step is submitted. It declares which task runs next.
 - Inputs can be plain text, number, multi select, single select

- **A branching task** resolved by the engine with no human action. It picks the next task based on data collected earlier in the run (e.g. route one way if an entered amount exceeds a threshold, another way otherwise), with a fallback path when nothing matches.
- If `task_a.amount > 500` then go to `task_b` or `task_c`
- **A terminal task** that ends the run.

Running a workflow (the instance)

3. A user can start a run from a definition.
4. A user can submit data to the current data-entry task. Submission supports two modes: saving progress without finishing, and finishing the current step. Finishing the final step completes the task.
5. When a task completes, the engine advances the run on its own until end is reached
6. Data collected by completed tasks is retained for the life of the run and is what branching tasks consult.
7. A user can inspect a run: its overall status, and its tasks separated into what's done and what's currently active.
8. A user can cancel a run that is still in progress.

Behavioural rules

9. A run started from a definition is unaffected by later edits to that definition (the run executes against the flow as it was when started). State how you achieve this.
10. Branching must be data-driven and demonstrable: the **same** definition, started twice with different inputs, must take **different** paths and both reach completion.

4. Explicitly out of scope

So you can spend your time on the engine, ignore: authentication/authorization, file uploads, e-signatures, notifications/emails, audit trails, analytics, parallel/concurrent branches that fan out and join, deadlines/timeouts, and any task kinds beyond the three above. Mentioning how you'd accommodate them later is welcome; building them is not.

5. Deliverable

A repo or archive containing:

- The service, runnable with a single documented command.

- A **README** covering: how to run it; the API contract (shapes + example calls); and — for the branching task — exactly what its decision logic can express and how it's written in a definition.
 - A **worked example**: a definition containing a branch, and the sequence of API calls that drives a run down each side of that branch to completion.
-

7. Optional stretch (pick at most one)

- Asynchronous orchestration of the workflow ; explain how you keep the run consistent.
- Make concurrent submissions to the same active task safe, so two racing requests can't both succeed.
- Introduce a fourth kind of task (parallel task) and show it slotting into your model with minimal disturbance to the core.